

How do different agent harnesses affect the cost-efficiency and task-solving performance of coding models on software-engineering tasks?

Fares Rahmani

Abstract TODO: Write the abstract after the study is complete.

Table of contents

1	Introduction	5
1.1	Problem statement	5
1.2	Research question	5
1.3	Objective	5
1.4	Scope	5
2	Current State in the Company	5
2.0.a	Internal Survey	5
2.0.b	Agent Harness Usage	5
2.0.c	Model Usage	6
2.0.d	Reported Cost	6
3	Methodology	6
3.1	Benchmark and task selection	6
3.2	Model and harness selection	7
3.3	Experimental controls	7
3.3.a	Model profiles	8
3.3.b	Model routing	9
3.3.c	Provider concurrency	9
3.4	Benchmark infrastructure	9
3.4.a	OpenCode V2 integration	10
3.5	Compatibility interventions	11
3.5.a	Codex with third-party models	11
3.6	Cost accounting	11
3.7	Reproducibility	12
3.8	Historical 2026-08-31 Kimi K3 batch	12
4	Results and Analysis	13
4.1	Task success	13

4.2 Cost	13
4.3 Token usage	13
4.4 Cache usage	13
4.5 Runtime	13
4.6 Comparative analysis	13
5 Conclusion	13
5.1 Answer to research question	13
5.2 Limitations	13
5.3 Outlook	13
Bibliography	14

List of Figures

List of Tables

1 Introduction

Placeholder section: Replace the bullets below with the paper's content. No findings, company facts, or sources have been added yet.

1.1 Problem statement

- TODO: State the problem to be investigated.

1.2 Research question

- TODO: Formulate the research question.

1.3 Objective

- TODO: State the objective of the Praxisarbeit.

1.4 Scope

- TODO: Define the scope and boundaries of the work.

2 Current State in the Company

draft

2.0.a Internal Survey

To assess the current use of AI coding agents, an internal survey was conducted between August and September 2026. The survey received 56 responses. Participation was voluntary and the sample was not selected to be representative of the company as a whole. The results are therefore used descriptively to identify usage patterns among the respondents rather than to estimate company-wide adoption rates.

2.0.b Agent Harness Usage

Claude Code was clearly the dominant agent harness among respondents. It was selected as the first-choice harness by 40 of 56 respondents (71.4%) and appeared in the weekly workflow of 51 respondents (91.1%). OpenCode was the second-most common first choice with 8 respondents (14.3%) and was used weekly by 14 respondents (25.0%). Codex and GitHub Copilot were each used weekly by 9 respondents (16.1%), while Pi was used by 5 respondents (8.9%).

Despite the concentration around Claude Code, the reported workflows were not homogeneous. A total of 41 respondents (73.2%) stated that they used at least one additional AI coding setup or model each week, and 31 respondents (55.4%) reported using two or more distinct harnesses. The most common multi-harness combination was Claude Code together with OpenCode, reported by 11 respondents. Claude Code was also frequently combined with GitHub Copilot and Codex.

2.0.c Model Usage

Anthropic models were similarly dominant. An Anthropic model was selected as the primary model by 41 respondents (73.2%), while 50 respondents (89.3%) reported using at least one Anthropic model in their weekly workflow.

Claude Opus 5 was the most common primary model, selected by 20 respondents (35.7%), followed by Claude Sonnet 4.6 with 10 respondents (17.9%) and Claude Sonnet 5 with 8 respondents (14.3%). Looking at all reported weekly configurations, Opus 5 was used by 26 respondents (46.4%).

The most common primary harness-model configuration was **Claude Code with Claude Opus 5**, used by 18 respondents (32.1%). It was followed by Claude Code with Sonnet 5 at 14.3% and Claude Code with Sonnet 4.6 at 12.5%. When secondary setups are also considered, Claude Code with Opus 5 appeared in the workflows of 24 respondents (42.9%).

Non-Anthropic models were less common but showed greater variety. GPT-5.6 Sol was used weekly by five respondents, DeepSeek V4 Flash 0731 by four, and Kimi K3 by three. Overall, 23 respondents (41.1%) reported named models from at least two different model-provider families.

2.0.d Reported Cost

The cost responses were more difficult to compare because the questionnaire allowed both monetary ranges and answers such as flat-rate subscriptions or unknown costs. Of the 32 respondents who provided a monetary range, 21 (65.6%) reported monthly costs of at least EUR 100. However, these values should not be interpreted as an estimate of average company expenditure.

Overall, the survey indicates a **dominant Claude Code and Anthropic baseline combined with substantial experimentation with alternative harnesses and models**. This makes harness choice relevant even in an environment where one ecosystem is clearly preferred, since a considerable share of respondents already use multiple agent configurations in parallel.

3 Methodology

This study evaluates coding-agent harnesses as part of a complete model-harness system. The central experimental principle is to preserve behavior that belongs to the harness while controlling external differences that would otherwise distort the comparison. Native capabilities such as delegation and provider-specific compaction therefore remain enabled where supported, whereas routing, context limits, model selection, and usage accounting are constrained when necessary to keep measurements comparable.

3.1 Benchmark and task selection

DeepSWE v1.1 was selected during planning in July–August 2026 because it was a current public benchmark explicitly designed to distinguish frontier coding agents on original, long-horizon software-engineering tasks. Its grading also fits this study: submissions are evaluated by isolated functional verifiers with a binary pass/fail reward rather than by an LLM judge

[1]. This provides a reproducible success criterion without constructing and validating a new internal task set.

Running the complete benchmark across all model–harness combinations was outside the available budget. A deterministic subset of ten tasks was therefore selected from the public DeepSWE v1.1 results. For each programming language, one hard and one medium task was selected. Difficulty is derived from the solve rate of valid historical `mini-swe-agent` runs. The reference panel excludes Claude Opus 5, GPT-5.6 Luna, DeepSeek V4 Flash, and Kimi K3 because these were the four model configurations considered benchmark targets when the task sample was selected on 2026-08-18. Their own public DeepSWE results were therefore not allowed to influence the task difficulty or token-intensity values used for sampling. The final model matrix changed later; this exclusion set is part of the historical sampling procedure rather than a description of the final evaluated models. Hard tasks occupy the 75th–100th difficulty percentile, including exactly 75, while medium tasks occupy the 50th percentile up to but excluding 75. Within each language and difficulty stratum, the task with the highest median historical token usage is selected. This deliberately favors demanding tasks on which differences in context handling and token efficiency are more likely to become visible.

The selection was run on 2026-08-18 using the then-current public DeepSWE v1.1 API responses. `data/deepswe_task_selection_v1.1.json` records the resulting task IDs together with SHA-256 hashes of the `tasks.json`, `trials.json`, and `release.json` responses used at that time. The committed task IDs define the fixed task set for the experiment; the hashes provide provenance for the historical sampling inputs. Reproducing the benchmark therefore uses the committed task set rather than rerunning the selection against later API data. The complete selection procedure is implemented in `scripts/select_deepswe_tasks.py`.

3.2 Model and harness selection

The harness set consists of Claude Code, OpenAI Codex CLI, Pi, and OpenCode V2. Claude Code and Codex represent first-party coding agents from major model providers. OpenCode was the second-most common first-choice harness in the internal survey described in Section 2, while Pi provides a comparatively lightweight alternative. OpenCode V2 was selected instead of V1 because it is the current major redesign and is more representative of the tool’s future development.

The model set spans several providers and price classes: Claude Opus 5 at medium reasoning, and GPT-5.6 Luna, Kimi K3, DeepSeek V4.1 Flash, and GLM-5.3-Flash at max reasoning. The purpose is not to identify the strongest model in isolation, but to determine whether harness behavior remains effective across different model families.

3.3 Experimental controls

Each trial evaluates exactly one model through one harness. Harness-native delegation remains enabled because subagent orchestration is part of the behavior under study. However, delegated calls are constrained to the same benchmark model as the parent session. This prevents a nominally single-model trial from becoming a multi-model ensemble. The Pier

adapters aggregate usage across the complete parent/child session tree while retaining the root session as the main trajectory.

Provider-specific harness optimizations are otherwise retained. For example, a harness may use its native compaction or caching behavior when the selected model supports it. Equivalent features are not recreated through custom plugins in competing harnesses, because the objective is to evaluate the harnesses as practical systems rather than force their internal implementations to be identical.

GPT-5.6 Luna supports a larger API context window, but the first-party Codex 0.151.0 model profile uses 272,000 tokens as its default context window and requires an explicit override to use more [2]. OpenAI also applies higher pricing when a request exceeds 272,000 input tokens [3]. The experiment therefore adopts the same 272,000-token operating window recommended by the first-party Codex default for every harness. Allowing only some harnesses to use the larger window would make cost depend on inconsistent context configuration and could move one harness into a different pricing tier.

External web-search capability is disabled in the benchmark configuration wherever the harness exposes a controllable search tool. This keeps trials limited to the repository, benchmark environment, and harness-native development tools rather than allowing external information retrieval.

3.3.a Model profiles

Where a harness ships a provider or model profile for a tested model, that profile is used as the base configuration. Only settings required to reach the designated inference endpoint are changed, such as the base URL, credentials, or gateway-facing model identifier. Harness-supplied compatibility settings, reasoning mappings, context and output limits, and other model-specific behavior are retained unless they conflict with an experimental control defined in this chapter. A custom profile is introduced only when the pinned harness version has no usable entry for the tested model or when the selected provider route requires narrowly scoped transport metadata.

Pi follows this policy for the tested open-weight models. Kimi K3 retains Pi 0.84.4's bundled Moonshot/Kimi metadata while only the provider endpoint and API key are changed. GLM-5.3-Flash retains the bundled profile except for `compat.thinkingFormat`. Pi's Z.AI request path sends `thinking` and, when enabled, `reasoning_effort` together [4]. The Fireworks-backed GLM route rejected this pair during acceptance testing, so the benchmark uses `thinkingFormat: "openai"` and sends only `reasoning_effort`. The reasoning level and model limits remain unchanged. OpenCode V2 follows the same profile policy: built-in catalogue entries are reused where possible, with explicit metadata only for missing models or required transport compatibility.

Codex 0.151.0 ships a native profile for GPT-5.6 Luna, so Luna uses that profile with only the endpoint and authentication path redirected [2]. The same Codex release does not ship native profiles for Claude Opus 5, Kimi K3, DeepSeek V4.1 Flash, or GLM-5.3-Flash. Unknown

models instead receive a minimal fallback descriptor without configured multi-agent mode or tool mode, model-specific reasoning levels, and several other profile features available to recognized models [5]. Using this fallback would therefore evaluate those models with a substantially reduced Codex capability set.

For Claude Opus 5, Kimi K3, DeepSeek V4.1 Flash, and GLM-5.3-Flash, the experiment derives Codex metadata from the frozen GPT-5.6 Sol profile in Codex 0.151.0 and replaces the model-specific identity, context, modality, reasoning, and compatibility fields required for each route. These entries use Multi-Agent V1 and disable Responses Lite for the non-OpenAI routes while retaining the remaining Sol-derived behavior. This is a documented profile intervention rather than a neutral configuration choice: it is used so that these models are evaluated with the broader Codex agent feature set available to recognized models instead of the intentionally minimal unknown-model fallback. Codex results for these models are therefore conditional on this profile configuration.

3.3.b Model routing

All non-Anthropic model traffic is routed through AiOrbit, the internal LiteLLM proxy used by the organization. The open-weight models are served through Fireworks behind this proxy. Anthropic models are the exception and call the Anthropic API directly. This routing path is kept constant for a given model across harnesses.

AiOrbit and Fireworks are infrastructure outside the experimental manipulation. Either may transform requests or introduce provider-side failures that the study cannot control. The results therefore characterize the harnesses under this fixed deployment path rather than an abstract provider-independent environment.

3.3.c Provider concurrency

Fireworks-backed jobs use model-specific concurrency caps so provider throttling does not become an experimental variable [6]. The Pi/Claude Code/Codex jobs run up to 30 Kimi K3 trials, eight GLM-5.3-Flash trials, and four DeepSeek V4.1 Flash trials concurrently. OpenCode V2 contains ten trials per model and runs ten Kimi or GLM trials concurrently, but five for DeepSeek. These settings affect job scheduling only; task definitions, model settings, scoring, and reported metrics are unchanged. The exact values are frozen in the benchmark configuration.

3.4 Benchmark infrastructure

The experiment does not use upstream Pier unchanged. It uses the project-specific [FZR-forks/pier](#) fork, which has diverged materially from the upstream runner used by DeepSWE [7], [8]. The fork became necessary because the selected model-harness matrix required additional adapters, stricter model isolation, and more complete usage accounting than upstream Pier exposed at the time of the experiment.

The fork adds Pi and OpenCode V2 support, complete delegated-session accounting for Claude Code and Codex, hard same-model enforcement for delegated calls, corrections for Codex

cumulative token snapshots and inherited child history, cache-write-aware token and cost accounting, support for the internal CA required to reach AiOrbit, and preservation of failed retry-attempt evidence. Some Claude Code trajectory changes were selectively backported from Harbor; other changes are specific to this experiment. As a result, the PA1 runner is substantially more capable for this benchmark matrix than the upstream Pier checkout and must be treated as a distinct, versioned part of the experimental apparatus rather than as stock Pier.

These changes were kept outside the benchmark task itself. They affect agent installation, routing, model enforcement, trajectory conversion, and measurement, but do not modify the DeepSWE task description, repository, verifier, or reward calculation presented to the agent.

3.4.a OpenCode V2 integration

OpenCode V2 is a complete rewrite of V1 and exposes a different server and session API. The existing V1 adapter therefore cannot drive V2. PA1 implemented a separate `opencode-v2` Pier adapter for the benchmark [8]. The benchmark freezes OpenCode V2 at version 2.0.8 after validating the packaged binary and the new adapter together. PA1 keeps the release version, platform-specific archive checksums, frozen model catalogue, and provider configuration in the benchmark configuration instead of hardcoding them into Pier. This separation keeps the adapter generic across V2 releases.

The adapter starts a private authenticated OpenCode server for every trial and never shares that server between trials. After the agent finishes, the adapter waits for the session tree to settle and collects paginated root, delegated, utility, and compaction-session records. It reconstructs recursive usage totals from these records without counting replayed messages twice. If collection remains incomplete, or if the server terminates unexpectedly, the adapter keeps the available trajectory and error evidence but does not report complete token and cost aggregates.

OpenCode’s native delegation remains enabled because delegation is part of the harness behavior under test. OpenCode 2.0.8 allows a subagent to select any model exposed by the catalogue, so root-agent pinning alone does not enforce a single-model trial. PA1 therefore disables catalogue fetching, loads a frozen catalogue narrowed to the selected model and reasoning variant, and enforces that selection for root, delegated, utility, and compaction requests. This control preserves OpenCode’s native multi-agent behavior while preventing a benchmark trial from silently using another model.

PA1 also keeps model identity separate from transport configuration. Luna keeps OpenCode’s built-in OpenAI Responses profile so that provider-specific behavior such as native compaction remains available. Kimi K3, DeepSeek V4.1 Flash, and GLM-5.3-Flash keep their canonical OpenCode model identities, while the configuration redirects only the transport package, gateway endpoint, and provider-facing model identifier to the fixed Fireworks-backed route. GLM additionally marks Fireworks as the transport-level canonical provider because OpenCode’s Z.AI transform otherwise adds request fields that the Fireworks route

rejects. These transport changes solve compatibility problems without replacing the model with a synthetic provider profile.

Two additional controls keep OpenCode comparable with the other harnesses. First, OpenCode does not reliably project its configured output limit into OpenAI-compatible requests, so PA1 injects the model-specific output ceiling directly into the request body [9]. Second, PA1 applies the same context and reasoning policy used by the other harnesses: primary Kimi, DeepSeek, GLM, and Luna runs use the configured `max` reasoning variant, Opus uses `medium`, and Luna uses the common 272,000-token operating window. PA1 does not add a custom retry algorithm to OpenCode. OpenCode keeps its native request retry behavior, while Pier may repeat a complete trial only for transport or gateway failures.

PA1 validates every primary OpenCode configuration with the pinned real binary before admitting it to the benchmark. The acceptance path exercises both Chat Completions and Responses transports, explicit output caps, model restriction, usage reconciliation, native compaction, simultaneous-trial isolation, and a dedicated delegation smoke test. The smoke jobs use low reasoning and small output limits only to validate the integration; they do not change the primary benchmark settings.

3.5 Compatibility interventions

Compatibility changes are permitted only when they are required to make a harness communicate correctly with the intended model endpoint. Such changes are kept narrowly scoped and are not used to add capabilities to a harness.

3.5.a Codex with third-party models

Codex 0.151.0 uses the OpenAI Responses protocol, while the Fireworks-backed models are reached through AiOrbit using Chat Completions. Direct translation through the LiteLLM gateway was not sufficient for Codex’s request and reasoning semantics. A pinned CLIProxyAPI instance is therefore used as a protocol bridge:

`Codex -> Responses -> CLIProxyAPI -> Chat Completions -> AiOrbit -> Fireworks.`

The bridge performs protocol translation only. Its own retries, credential cooldowns, and model fallbacks are disabled so it cannot improve the measured harness behavior.

During the benchmark period, an upstream CLIProxyAPI defect was identified in which a streamed chunk containing both `reasoning_content` and visible `content` could produce an invalid Responses item order [10]. Rather than upgrade the complete bridge and introduce unrelated changes, the benchmark keeps the frozen v7.2.146 base and backports only the upstream fix [11]. Logs from the completed 2026-08-31 Kimi K3 trial runs showed that the defect’s triggering condition did not occur, so those runs were retained.

3.6 Cost accounting

Cross-harness cost is calculated from recorded token and cache usage using a frozen pricing table in `benchmark/pricing.yaml`. Official model API prices are used rather than the invoice

price of the underlying proxy or inference host. This separates model and harness efficiency from provider-specific discounts or surcharges.

If a trial is discarded and repeated because of a transport or gateway failure, tokens consumed by the discarded attempt remain part of the reported cost. Removing them would systematically favor configurations that fail more often. In contrast, benchmark task timeouts are treated as outcomes of the evaluated system rather than infrastructure failures and are not automatically rerun.

3.7 Reproducibility

The selected DeepSWE task IDs are fixed in `data/deepswe_task_selection_v1.1.json`, while the runnable benchmark software is pinned separately. The standard benchmark wave uses DeepSWE revision `0b9fabbb63b9104d678fe965e1632f2dd9eaa2ea`, Pier revision `b0acdae033425500e968ac917acffeafd35f9cdb`, Codex CLI 0.151.0, Claude Code 2.1.251, Pi 0.84.4, and OpenCode V2 2.0.8. OpenCode additionally pins the platform-specific package checksums and the frozen 2.0.8 model catalogue. The compatibility-bridge image digest, model profiles, pricing table, generated configurations, concurrency settings, and task-selection audit data are stored with the benchmark setup. Pier writes a `lock.json` into each result directory so that every run can be traced to the exact configuration used. All model runs in this standard wave use the same frozen runner and harness setup.

3.8 Historical 2026-08-31 Kimi K3 batch

The Kimi K3 Pi, Claude Code, and Codex runs completed on 2026-08-31 are the only retained exception to the standard configuration above. They were completed before the final web-search policy and before the OpenCode V2 adapter was added, so they are documented separately rather than making their older setup appear to be the baseline for the rest of the experiment.

The first difference concerns the web-search tool surface. The 2026-08-31 Kimi configurations did not explicitly remove search-related tools. To preserve that tool surface, the later OpenCode V2 Kimi configuration also exposes OpenCode’s native web-search tool to the model. This does not provide functional Internet search in the benchmark: the configured route does not provide a working search backend, so the tool cannot retrieve external web content. The exception therefore concerns the tool interface visible to the model, not actual web access. The retained 2026-08-31 logs also contain no completed Internet-backed search. All standard-wave model configurations disable web-search tools explicitly.

The second difference is the Pier revision. The 2026-08-31 runs used `ff65bae55c9a8ff15ddd3c2967c81a936713dd4d`, while the standard wave uses `b0acdae033425500e968ac917acffeafd35f9cdb`. The later revision adds the independent OpenCode V2 adapter, its registration, and its tests. It does not change the existing Pi, Claude Code, Codex, or OpenCode V1 adapters. The only shared installed-agent change adds a logging hook that removes credential values from debug metadata without changing the environment passed to the agent process. The historical Kimi runs use Pi, Claude Code, and Codex, so these

later Pier changes do not affect their task execution or model selection and should not affect their accounting. The experiment therefore retains the completed Kimi measurements instead of rerunning them only to align the runner commit.

4 Results and Analysis

Placeholder section: Do not add results until the underlying data and analysis are available and documented.

4.1 Task success

- TODO: Report task-success results from the recorded data.

4.2 Cost

- TODO: Report cost results from the recorded data.

4.3 Token usage

- TODO: Report token-usage results from the recorded data.

4.4 Cache usage

- TODO: Report cache-usage results from the recorded data.

4.5 Runtime

- TODO: Report runtime results from the recorded data.

4.6 Comparative analysis

- TODO: Compare the results using the defined methodology and metrics.
- TODO: Add executable Quarto figure cells here once the benchmark result data is frozen.
See [scripts/README-figures.md](#) for the Pareto-frontier and line-chart patterns.

5 Conclusion

Placeholder section: Add conclusions only after the research question has been addressed by the documented analysis.

5.1 Answer to research question

- TODO: Answer the research question based on the results.

5.2 Limitations

- TODO: Describe methodological and practical limitations.

5.3 Outlook

- TODO: Identify appropriate future work.

Bibliography

- [1] W. Huang, C. Lee, L. Tng, and S. Ge, “DeepSWE: Measuring Frontier Coding Agents on Original, Long-Horizon Engineering Tasks,” *arXiv preprint arXiv:2607.07946*, 2026, Accessed: Sep. 15, 2026. [Online]. Available: <https://arxiv.org/abs/2607.07946>
- [2] OpenAI, “Codex model catalog for release 0.151.0.” Accessed: Sep. 15, 2026. [Online]. Available: <https://github.com/openai/codex/blob/rust-v0.151.0/codex-rs/models-manager/models.json>
- [3] OpenAI, “GPT-5.6 Luna Model.” Accessed: Sep. 15, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
- [4] Pi contributors, “OpenAI Completions adapter in Pi 0.84.4.” Accessed: Sep. 20, 2026. [Online]. Available: <https://github.com/earendil-works/pi/blob/v0.84.4/packages/ai/src/api/openai-completions.ts>
- [5] OpenAI, “Codex fallback model metadata for release 0.151.0.” Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/openai/codex/blob/rust-v0.151.0/codex-rs/models-manager/src/model_info.rs
- [6] Fireworks AI, “Rate limits.” Accessed: Sep. 20, 2026. [Online]. Available: <https://docs.fireworks.ai/serverless/rate-limits>
- [7] DataCurve, “Pier.” Accessed: Sep. 15, 2026. [Online]. Available: <https://github.com/datacurve-ai/pier>
- [8] FZR-forks, “Pier: PA1 experimental fork.” Accessed: Sep. 15, 2026. [Online]. Available: <https://github.com/FZR-forks/pier>
- [9] TheFerragamo, “v2 (opencode2): max_tokens never sent for @ai-sdk/openai-compatible models, limit.output ignored, thinking turns truncate at provider default.” Accessed: Sep. 15, 2026. [Online]. Available: <https://github.com/anomalyco/opencode/issues/47398>
- [10] MAXOUXAX, “openai-compatibility: Codex shows a truncated assistant message when one chunk holds content and reasoning_content (deepseek-v4-flash).” Accessed: Sep. 14, 2026. [Online]. Available: <https://github.com/router-for-me/CLIProxyAPI/issues/5659>
- [11] L. Pater, “fix(openai): process reasoning deltas before content in responses stream.” Accessed: Sep. 14, 2026. [Online]. Available: <https://github.com/router-for-me/CLIProxyAPI/commit/c8ecb4f3c972664aae802e21c6a6743d5f6bd80a>